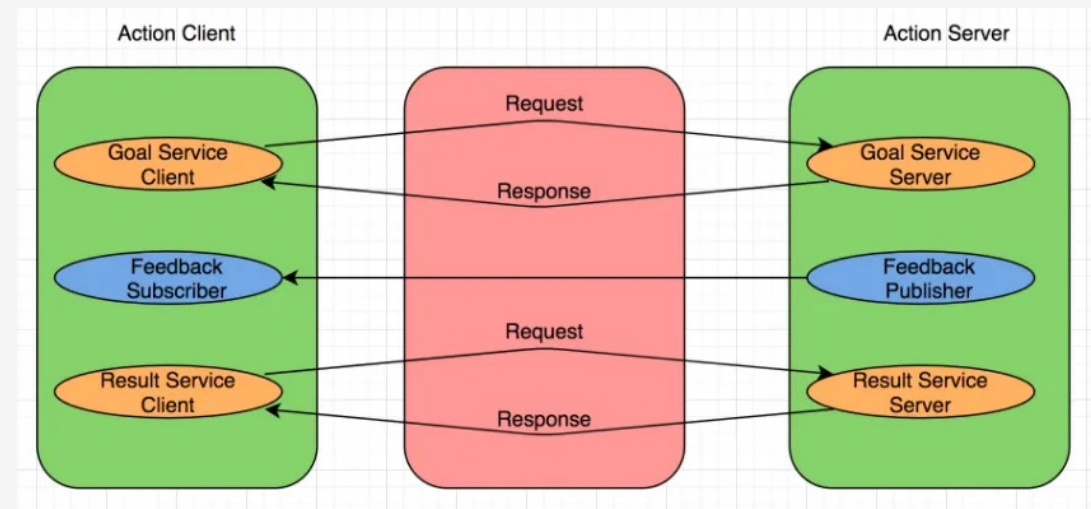


Action 개요

Action

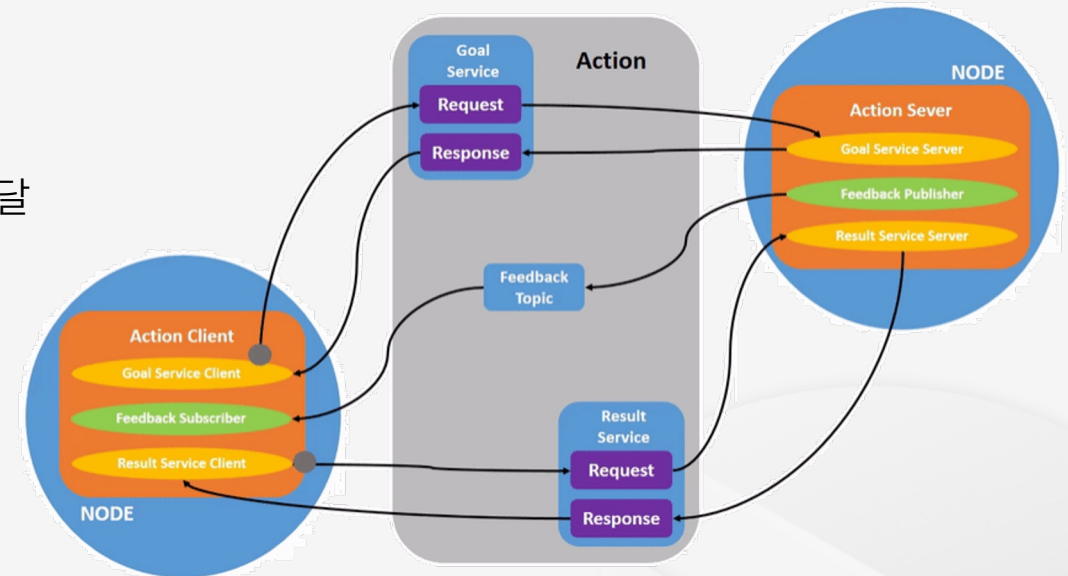
Action이란?

- 장기 실행 작업을 위해 설계된 비동기 통신 방식
- 액션은 클라이언트(Client)와 서버(Server)로 구성
 - Action Server: 특정 액션을 제공하는 노드
 - Action Client: 액션을 요청하는 노드
- 액션에서 사용하는 세 가지 메시지 유형
 - **Goal**: Action Client가 서버에 요청하는 작업의 목표
 - **Feedback**: 작업이 진행 중일 때, Action Server가 Action Client에게 전달하는 중간 상태 정보
 - **Result**: 작업이 완료되면 Action Server가 Action Client에 반환하는 최종 결과



Action이란?

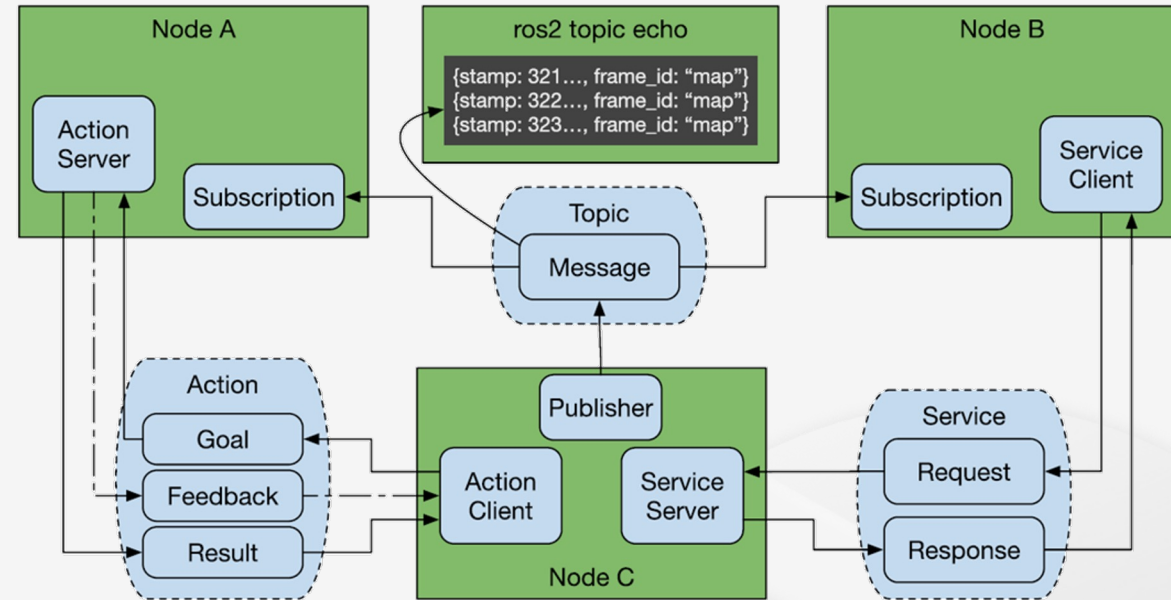
- 작동 원리
 - 액션 클라이언트가 **Goal**을 보내면, 액션 서버가 작업을 수행하면서 중간 중간 **Feedback**을 클라이언트에게 전송
 - 작업이 완료되면 액션 서버는 최종 **Result**를 액션 클라이언트에게 전달
 - 작업이 실행 중일 때, 액션 클라이언트가 원하면 작업 취소 가능
- 특징
 - 비동기적 처리
 - 진행 중인 작업 취소 가능
 - 피드백을 통한 진행 상황 모니터링



Interface Type 정의 (Action)

Interface란?

- ROS2에서 노드 간에 주고받는 데이터의 구조를 정의
- Interface는 .msg, .srv, .action 확장자를 가진 텍스트 파일로 정의
- 메시지 (msg): Topic을 통한 단방향 통신에 사용
- 서비스 (srv): 요청-응답 모델의 양방향 통신에 사용
- 액션 (action): 장기 실행 작업을 위한 비동기 양방향 통신에 사용



Interface Type 정의: Action

- 액션(Action)은 장시간 지속되는 작업을 수행할 때 사용되며, 중간에 상태를 모니터링하거나 작업을 중단할 수 있는 구조
- 주로 로봇이 특정 작업을 수행하는 동안 그 진행 상태를 모니터링하거나 취소할 수 있는 작업에 적합
- 액션 파일(.action)은 목표(goal), 피드백(feedback), 결과(result) 형식을 정의하는 파일

```
# Goal 부분
int32 order

---

# Result 부분
int32[] sequence

---

# Feedback 부분
int32[] sequence
```

액션 예시 (example_interfaces/Fibonacci.action)

Interface Type 비교

```
float64 x
float64 y
float64 z
```

메시지 예시 (geometry_msgs/Point.msg)

```
# Request 부분
bool data

---

# Response 부분
bool success
string message
```

서비스 예시 (std_srvs/SetBool.srv)

```
# Goal 부분
int32 order

---

# Result 부분
int32[] sequence

---

# Feedback 부분
int32[] sequence
```

액션 예시 (example_interfaces/Fibonacci.action)

Thank You